

NETWORK RECONNAISSANCE & PORT SCAN DETECTION

*Simulating, Investigating, and Detecting TCP Port Scanning
in a Windows Active Directory Lab*

Ryan Morales

Active Directory Home Lab

Splunk Enterprise | Kali Linux | Nmap | Windows Security Auditing
August 2026

1. Project Overview

The goal of this project was to simulate network reconnaissance by using Nmap from a Kali Linux virtual machine to scan a Windows workstation (WS01). Splunk was used as the SIEM to collect and analyze security telemetry generated by the scanning activity. After investigating the captured events, a detection rule was developed to identify port-scanning behavior and automatically generate an alert when the configured detection threshold was exceeded.

2. Lab Environment

System / Tool	Role
Kali Linux — 192.168.100.30	Served as the simulated attacker and generated network reconnaissance activity using Nmap against WS01.
WS01 — 192.168.100.20	Served as the target Windows workstation. Windows security auditing recorded blocked network traffic generated during the Nmap scans.
DC01 — 192.168.100.10	Served as the domain controller and hosted Splunk Enterprise, which was used to search, analyze, and detect security events forwarded from WS01.
Splunk Universal Forwarder	Forwarded Windows security telemetry from WS01 to Splunk Enterprise on DC01.
Windows Event ID 5152	Provided telemetry showing packets blocked by the Windows Filtering Platform.
Nmap	Generated the simulated network reconnaissance and port-scanning activity.

3. Attack Simulation

Network reconnaissance was simulated from the Kali Linux VM using Nmap. WS01 (192.168.100.20) was selected as the target, and TCP ports 1 through 1000 were scanned using:

```
sudo nmap -sS -Pn -p 1-1000 192.168.100.20
```

In this command, -sS performs a TCP SYN scan, -Pn skips host discovery and treats the target as online, and -p 1-1000 specifies the range of TCP ports to scan.

The purpose of the scan was to generate realistic port-scanning activity against the Windows workstation. This produced security telemetry that could then be investigated in Splunk and ultimately used to develop and validate a port-scan detection rule.

```

ryan@KALI01: ~
Session Actions Edit View Help

(ryan@KALI01)-[~]
$ sudo nmap -sS -Pn -p 1-1000 192.168.100.20
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-06 16:06 -0500
Nmap scan report for 192.168.100.20
Host is up (0.00095s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
135/tcp   open  msrpc
MAC Address: 08:00:27:89:82:73 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 5.38 seconds

(ryan@KALI01)-[~]
$

```

Figure 1 — TCP SYN port scan launched from Kali Linux against ports 1–1000 on WS01 (192.168.100.20). Nmap identified 999 filtered ports and TCP port 135 as open.

4. Splunk Investigation

In Splunk, I began the investigation after identifying that the Windows workstation was receiving a large number of blocked packets from an unknown source IP address, 192.168.100.30.

Using SPL searches, I isolated traffic from the source and analyzed the destination ports it was targeting. The investigation showed that the source was attempting connections across hundreds of different ports on WS01 within a very short period of time.

Further analysis identified 999 unique destination ports, a pattern consistent with automated port-scanning activity. These findings provided the basis for developing a detection rule to automatically identify similar reconnaissance behavior.

The screenshot shows the Splunk web interface with a search result for a blocked packet. The event details are as follows:

Type	Field	Value	Actions
Selected	host	WS01	
	source	WinEventLogSecurity	
	sourcetype	WinEventLogSecurity	
Event	Application_Name	-	
	ComputerName	WS01.home.local	
	Destination_Address	192.168.100.20	
	Destination_Port	27	
	Direction	Inbound	
	EventCode	5152	
	EventType	0	
	Filter_Origin	Stealth	
	Filter_Run_Time_ID	76446	
	Keywords	Audit Failure	
	Layer_Name	Transport	
	Layer_Run_Time_ID	13	
	LogName	Security	
	Message	The Windows Filtering Platform has blocked a packet. Application Information: Process ID: 0 Application Name: - Network Information: Direction: Inbound Source Address: 192.168.100.30 Source Port: 4702 8 Destination Address: 192.168.100.20 Destination Port: 27 Protocol: 6 Filter Information: Filter Origin: Stealth Filter Run-Time ID: 76446 Layer Name: Transport Layer Run-Time ID: 13	
	OpCode	Info	
	Process_ID	0	
	Protocol	6	
	RecordNumber	20495	
	SourceName	Microsoft Windows security auditing	
	Source_Address	192.168.100.30	

Figure 2 — Windows Event ID 5152 recorded on WS01 showing an inbound packet from 192.168.100.30 blocked by the Windows Filtering Platform.

New Search

index=main host=WS01 EventCode=5152 Source_Address="192.168.100.30" Destination_Address="192.168.100.20" | table _time Source_Address Destination_Address Destination_Port | sort - _time

1,998 events (8/10/26 7:33:00.000 PM to 8/10/26 7:48:00.000 PM) No Event Sampling

Events Patterns Statistics (1,998) Visualization

Show: 20 Per Page Format Preview: On

_time	Source_Address	Destination_Address	Destination_Port
2026-08-10 19:43:27.795	192.168.100.30	192.168.100.20	485
2026-08-10 19:43:27.795	192.168.100.30	192.168.100.20	327
2026-08-10 19:43:27.795	192.168.100.30	192.168.100.20	571
2026-08-10 19:43:27.792	192.168.100.30	192.168.100.20	974
2026-08-10 19:43:27.792	192.168.100.30	192.168.100.20	157
2026-08-10 19:43:27.792	192.168.100.30	192.168.100.20	568
2026-08-10 19:43:27.792	192.168.100.30	192.168.100.20	919
2026-08-10 19:43:27.792	192.168.100.30	192.168.100.20	393
2026-08-10 19:43:27.786	192.168.100.30	192.168.100.20	768
2026-08-10 19:43:27.786	192.168.100.30	192.168.100.20	466
2026-08-10 19:43:27.782	192.168.100.30	192.168.100.20	927
2026-08-10 19:43:27.782	192.168.100.30	192.168.100.20	388
2026-08-10 19:43:27.782	192.168.100.30	192.168.100.20	138
2026-08-10 19:43:27.777	192.168.100.30	192.168.100.20	206
2026-08-10 19:43:27.677	192.168.100.30	192.168.100.20	433
2026-08-10 19:43:27.677	192.168.100.30	192.168.100.20	561
2026-08-10 19:43:27.676	192.168.100.30	192.168.100.20	413
2026-08-10 19:43:27.676	192.168.100.30	192.168.100.20	24
2026-08-10 19:43:27.676	192.168.100.30	192.168.100.20	682

Figure 3 — Splunk investigation showing repeated blocked connections from 192.168.100.30 to numerous destination ports on WS01 (192.168.100.20).

5. Detection Development

After investigating the port-scanning activity, I developed an SPL detection rule to identify similar reconnaissance behavior automatically.

The detection groups blocked network traffic by source and destination IP address and calculates both the number of unique destination ports targeted and the total number of blocked packets.

Total_Blocked_Packets shows the total amount of blocked traffic, while Unique_Ports shows how many different destination ports were targeted by the source.

The detection uses a threshold of 10 or more unique destination ports to identify potentially suspicious scanning behavior. When a source reaches or exceeds this threshold, the activity is returned as a detection result for further investigation.

```
index=main host=WS01 EventCode=5152
| stats dc(Destination_Port) as Unique_Ports count as Total_Blocked_Packets by Source_Address
  Destination_Address
| where Unique_Ports >= 10
| sort - Unique_Ports
```

The screenshot shows the Splunk Enterprise interface with a search results table. The search bar contains a complex SPL query. The results table has four columns: Source_Address, Destination_Address, Unique_Ports, and Total_Blocked_Packets. One row is visible, showing Source_Address 192.168.100.30, Destination_Address 192.168.100.20, Unique_Ports 999, and Total_Blocked_Packets 1998.

Source_Address	Destination_Address	Unique_Ports	Total_Blocked_Packets
192.168.100.30	192.168.100.20	999	1998

Figure 4 — Tuned SPL detection identifying 192.168.100.30 targeting 999 unique destination ports on WS01, exceeding the configured threshold of 10 unique ports.

6. Alert Configuration

After validating the detection logic, I converted the SPL search into a scheduled Splunk alert. This removed the need to manually execute the detection search repeatedly and allowed Splunk to automatically evaluate network activity for the suspicious behavior identified during the investigation.

The alert was configured to run once per hour and evaluate the previous one hour of events (-1h to now).

If the detection returned activity involving 10 or more unique destination ports, Splunk generated a High-severity alert that could then be reviewed through the Triggered Alerts interface.

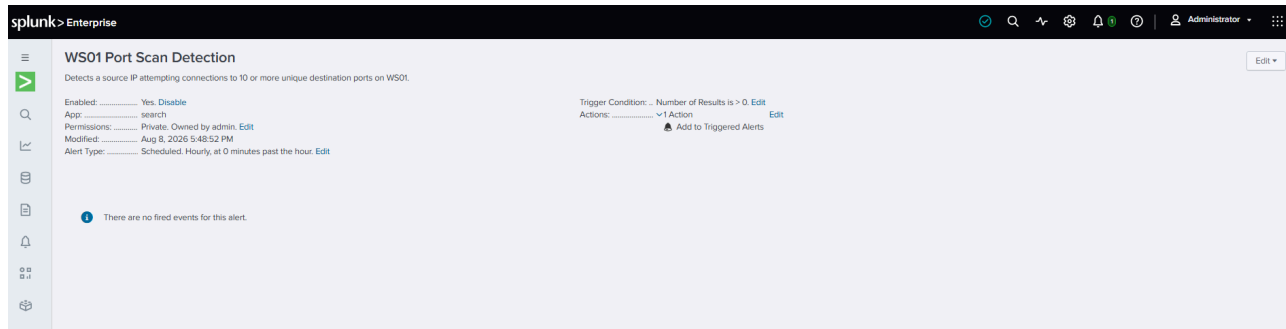


Figure 5 — WS01 Port Scan Detection configured as an enabled hourly scheduled alert in Splunk.

7. Detection Results

During final validation, Splunk detected suspicious network activity originating from 192.168.100.30 and targeting WS01 at 192.168.100.20.

999 unique destination ports

3,996 total blocked packets

Two TCP port scans were performed during the one-hour search window used by the scheduled alert. Because both scans targeted essentially the same range of ports, the unique destination-port count remained at 999. However, the total number of blocked packets increased because events from both scans were included in the detection window. This differed from the earlier detection test, which recorded 1,998 blocked packets from a single scan.

At the next scheduled execution, Splunk automatically evaluated the previous hour of telemetry, determined that the activity exceeded the configured threshold, and generated a High-severity WS01 Port Scan Detection alert.

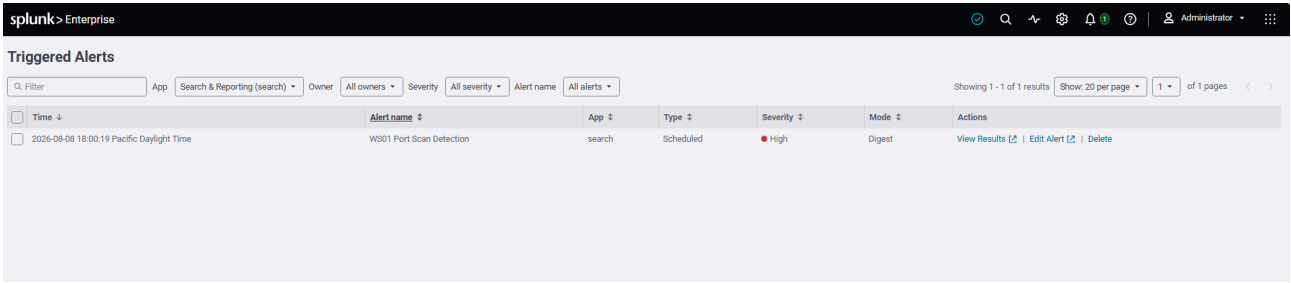


Figure 6 — High-severity WS01 Port Scan Detection successfully triggered after simulated port-scanning activity exceeded the configured detection threshold.

8. Skills Demonstrated & Lessons Learned

This project provided hands-on experience with network reconnaissance, SIEM investigation, and basic detection development. Through the project, I developed a stronger understanding of how to read and interpret security events in Splunk, analyze network activity using source and destination addresses and ports, and use SPL to filter and summarize security telemetry.

The project also strengthened my understanding of Nmap and how systems communicate across a network. Developing and testing the port-scan detection introduced me to the fundamentals of detection engineering, including establishing behavioral thresholds, aggregating events, scheduling searches, and validating alerts against simulated activity.

Troubleshooting throughout the project provided additional practical experience. Issues involving telemetry forwarding and inconsistent time-zone settings demonstrated the importance of maintaining a healthy logging pipeline and synchronized timestamps when performing security investigations.

While I am still developing my ability to independently write more complex SPL queries and detection rules, completing this project gave me practical experience with the detection-development process and a foundation that I can continue building on through future projects.